Mastering Augmented Reality with Unity

Prof. Jooho Cha, Ph.D.
jhcha@chungwoon.ac.kr

Dept. of Multimedia Science
Chungwoon University

Chapter 5. Creating Your First Custom Component

- Introduction
  - We will take a quick peek at C# within Unity, and see how we can use it to create a custom component.
  - We will also see how to communicate with other components and manipulate their values using C#.
  - If you are very new to programming and C#, it is recommended to go over the C# tutorials: https://docs.microsoft.com/en-us/dotnet/csharp/

Chapter 5. Creating Your First Custom Component

- Structure
  - Introduction to C# in Unity
  - Exploring the **MonoBehaviour**
  - Serialized variables
  - Debug class of Unity
  - Time in Unity
  - **GetComponent()**
  - Building a simple 3D clock

Chapter 5. Creating Your First Custom Component3D

- Objectives
  - We will be able to understand followings:
    - How C# scripts work with Unity
    - The core features of C# syntax with Unity
    - How to create custom components with C# language, and use those components to create a simple 3D clock

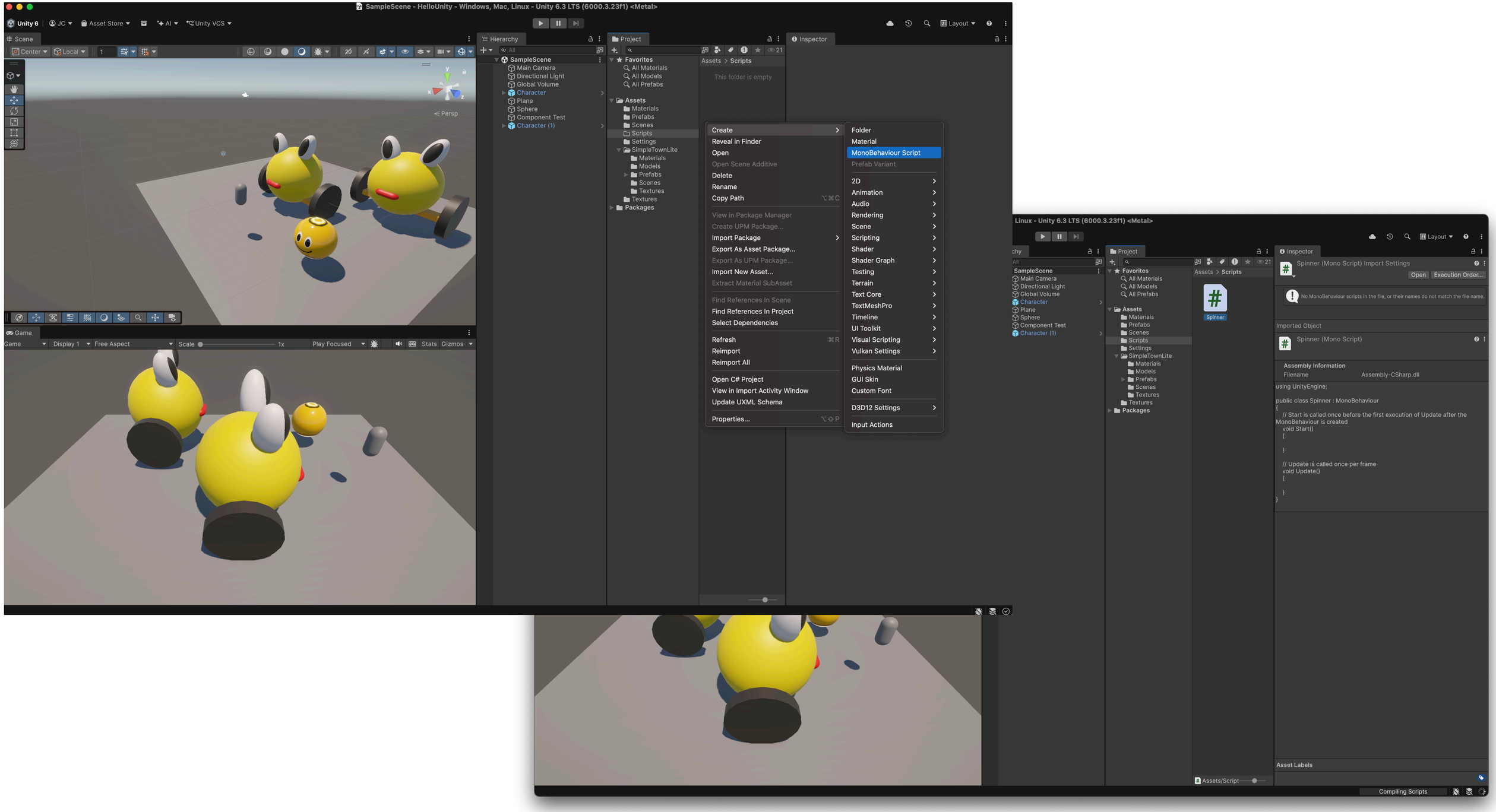
Introduction to C# in Unity

- In Unity, a C# script is considered as a component.
- A basic component script is a class file derived from **MonoBehaviour** class created by Unity.
  - Unity generates a class inherited from the **MonoBehaviour** class.
  - A C# script works like a blueprint for a custom component that can be attached to any game object in the scene.

Introduction to C# in Unity

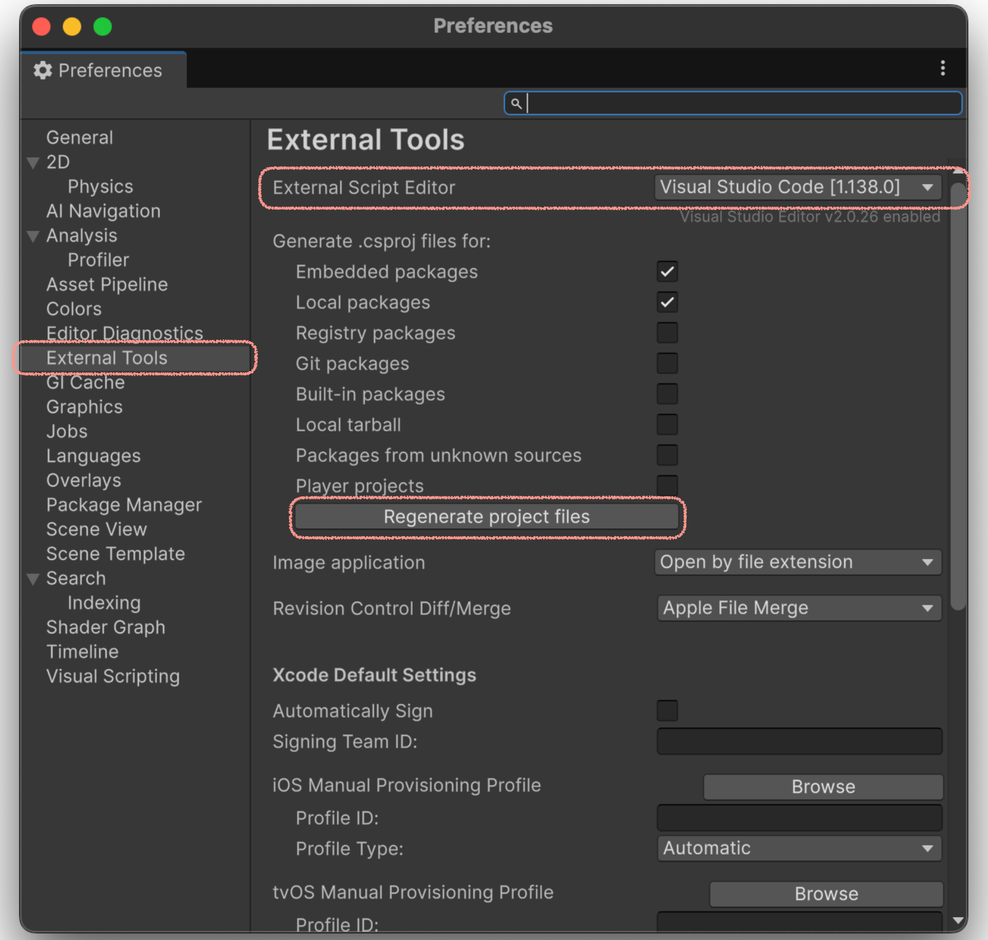
- Let us create our first component by following the given steps:

    1. Go to our SampleScene with the character and the smiley face sphere.
    2. Create a new folder called **Scripts** in the **Assets** folder.
    3. Inside the **Scripts** folder, right-click and click **Create** | **MonoBehaviour Script**.
    4. This will generate a default script called **NewMonoBehaviourScript**. We have to immediately rename it to **Spinner**.


SampleScene - HelloUnity - Windows, Mac, Linux - Unity 6.3 LTS (6000.3.23f1) <Metal>
Unity 6
Scene
Hierarchy
Project
Inspector
SampleScene
Main Camera
Directional Light
Global Volume
Character
Plane
Sphere
Component Test
Character (1)
Favorites
All Materials
All Models
All Prefabs
Assets
Materials
Prefabs
Scenes
Scripts
Settings
SimpleTownLite
Textures
Packages
This folder is empty
Persp
Game
Display 1
Free Aspect
Play Focused
Stats
Gizmos
Create
Reveal in Finder
Open
Open Scene Additive
Delete
Rename
Copy Path
View in Package Manager
Create UPM Package...
Import Package
Export As Asset Package...
Export As UPM Package...
Import New Asset...
Extract Material SubAsset
Find References In Scene
Find References In Project
Select Dependencies
Refresh
Reimport
Reimport All
Open C# Project
View in Import Activity Window
Update UXML Schema
Properties...
Folder
Material
MonoBehaviour Script
Prefab Variant
2D
Animation
Audio
Rendering
Scene
Scripting
Shader
Shader Graph
Testing
Terrain
Text Core
TextMeshPro
Timeline
UI Toolkit
Visual Scripting
Vulkan Settings
Physics Material
GUI Skin
Custom Font
D3D12 Settings
Input Actions
Linux - Unity 6.3 LTS (6000.3.23f1) <Metal>
Spinner (Mono Script) Import Settings
Open
Execution Order...
No MonoBehaviour scripts in the file, or their names do not match the file name.
Imported Object
Spinner (Mono Script)
Assembly Information
Filename
Assembly-CSharp.dll
using UnityEngine;
public class Spinner : MonoBehaviour
{
// Start is called once before the first execution of Update after the MonoBehaviour is created
void Start()
{
}
// Update is called once per frame
void Update()
{
}
}
Asset Labels
Assets/Script
Compiling Scripts

Introduction to C# in Unity

5. Make sure you have selected Visual Studio (Code) as the IDE for C# in Unity.
6. Go to **Edit** | **Preferences** (**Unity** | **Settings...** on Mac).
7. Select **External Tools** from the **Preferences** window.
8. For the **External Script Editor**, choose the Visual Studio (Code).
9. If you do not see the editor in the list, you can browse and choose.



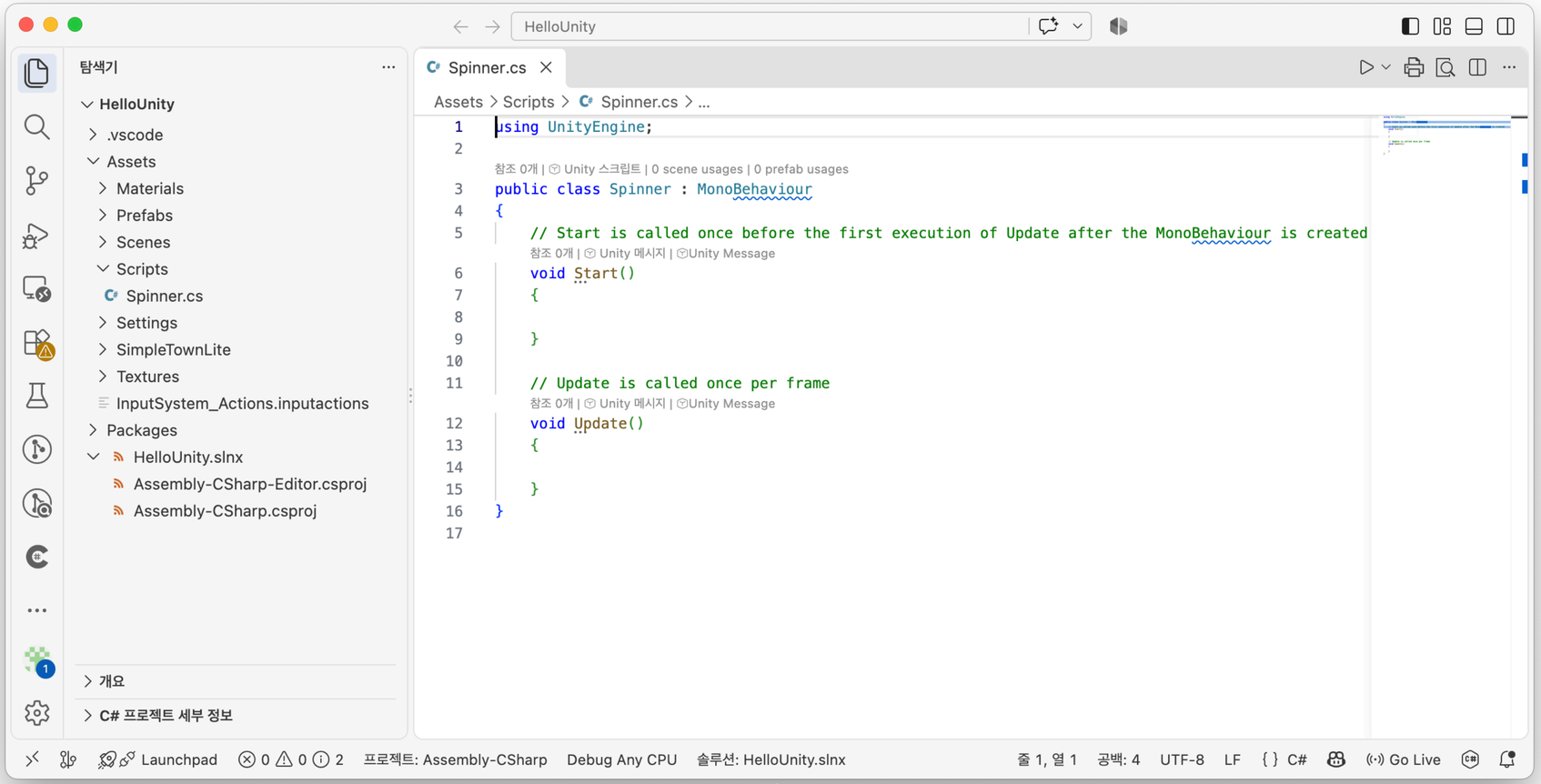
Introduction to C# in Unity

10. Click on the **Regenerate profile files** button. Now, the Unity editor is configured with Visual Studio (Code). This is only a one-time process.

11. Close the window, go to the **Scripts** folder under the **Assets**, and double-click on the script (**Spinner**) to launch it through Visual Studio (Code).

Exploring the MonoBehaviour

- As soon as you open the new C# script, this will be the template code you get:



Exploring the MonoBehaviour

- In Unity, the script must be attached to a game object to execute it.
  - The class name must be the same as the filename and is inherited from the **MonoBehaviour** component.
- There are two primary methods in any C# template script:
  - **Start()**: This method always executes only once at the very moment the game is started. Usually, it is used to initialize things in the scripts.
  - **Update()**: This is a loop method and executes once every frame. It can be used for any continuous checks, input detection, movement of objects, and so on.

Serialized variables

Serialized Variable(직렬화된 변수) ➜ C# 스크립트의 변수 값을
Unity Editor의 **Inspector 창에서 보고 수정할 수 있도록 만든 변수**

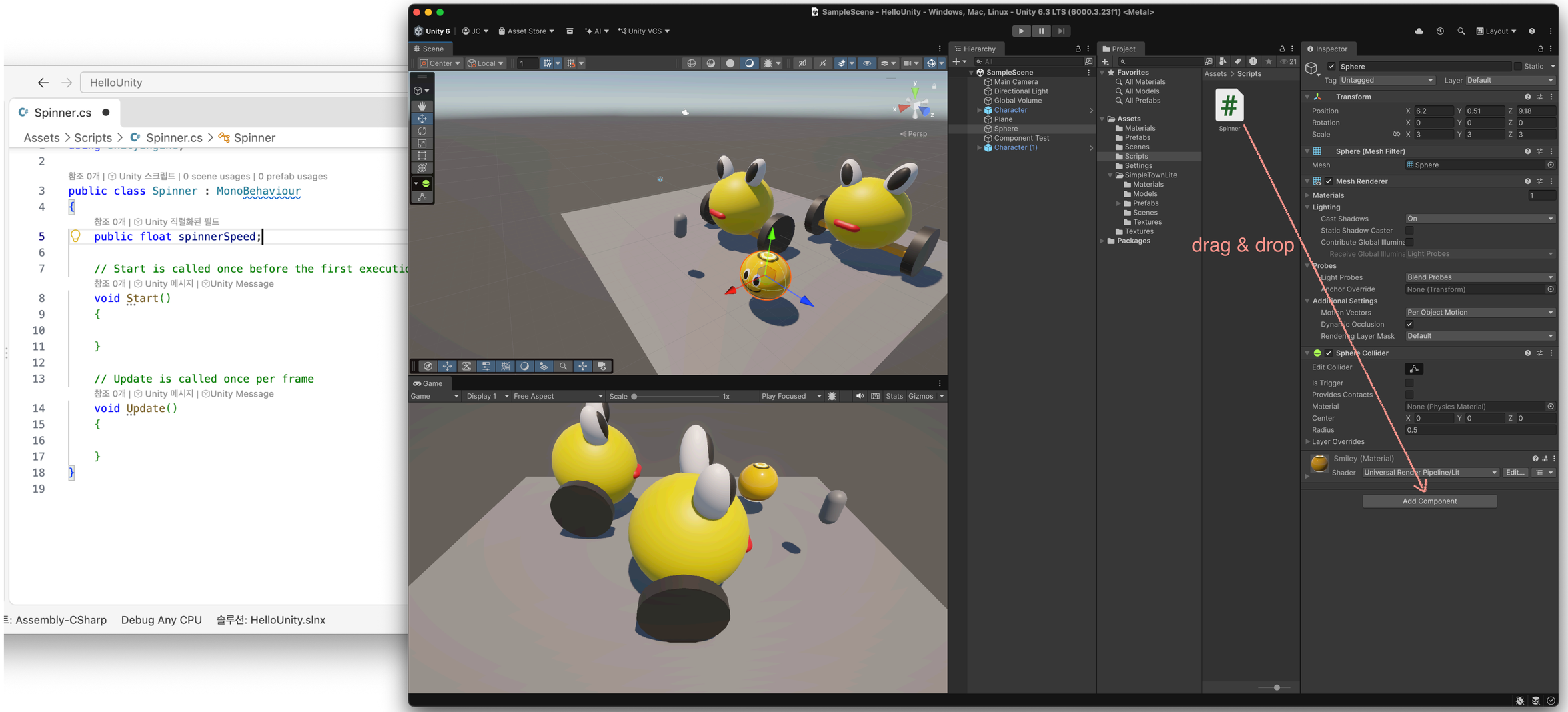
- Follow the given steps for serialized variables:

1. Before utilizing the **Start()** method, create a simple **public float** variable named as **spinnerSpeed** as follows:

   public float spinnerSpeed;

2. Now save the script and go back to Unity.
3. Select the smiley **Sphere** object we created before.
4. Drag and drop the **Spinner** script to the **Sphere** object.

Serialized variables

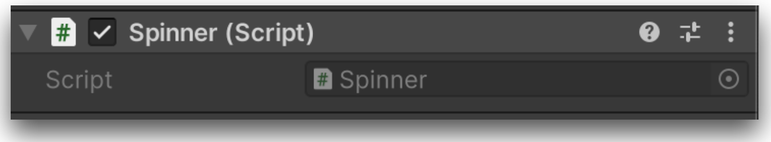


Serialized variables

5. Now you may see that a new custom component **Spinner** got attached to the **Sphere**, along with a text field to input the **Spinner Speed** value.

Serialized variables

6. Go back to the code, change **public** to **private**, save the script, and go back to Unity.
7. Select the **Sphere** and notice that the variable input is gone. This is because the access modifier is **private**.



8. We can use the **[SerializeField]** attribute in order to ask Unity to serialize it. This allows the Unity interface to show and modify the value. However, being a private variable, other scripts do not have the permission to the variable directly.

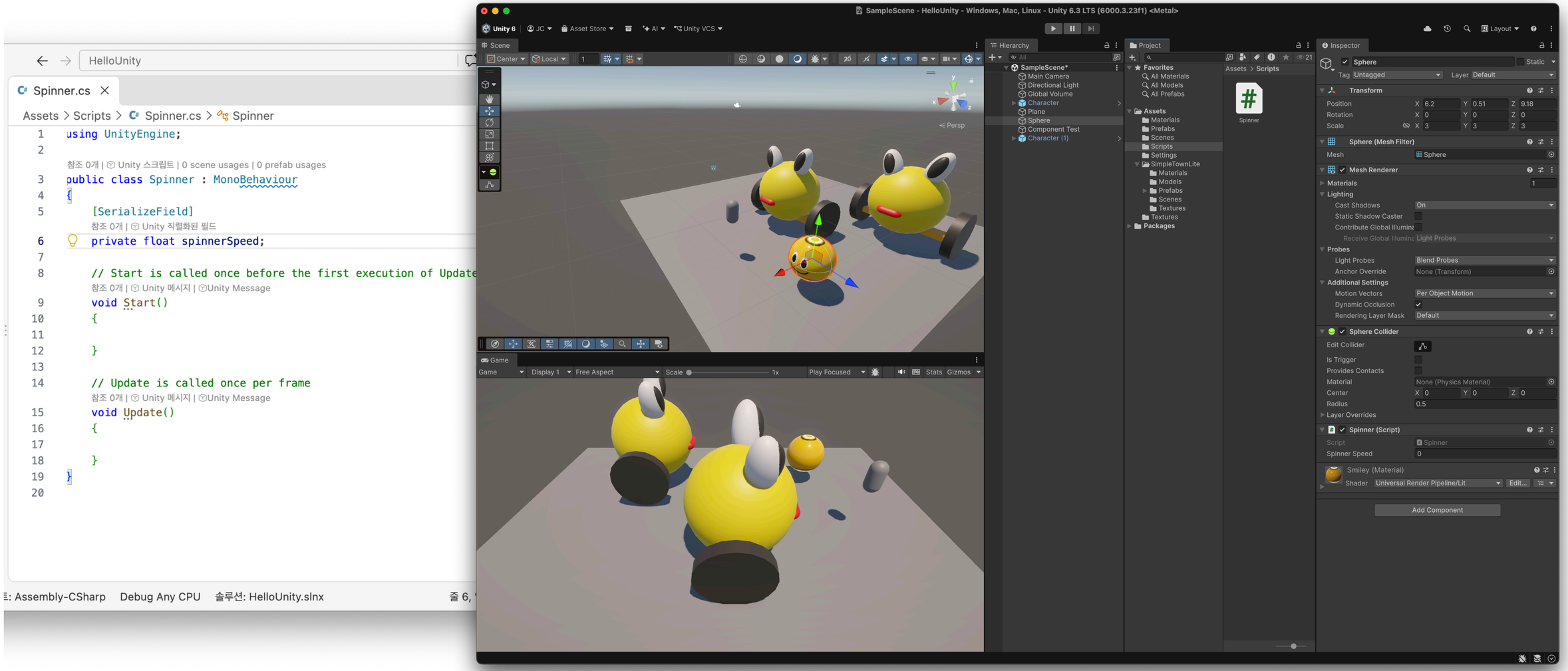
Serialized variables

9. Change the variable to a serialized field by adding the **[SerializeField]** attribute:

   [SerializeField] private float spinnerSpeed;

10. Save the script and go back to Unity. Notice that the field for **spinnerSpeed** is now back.

Serialized variables



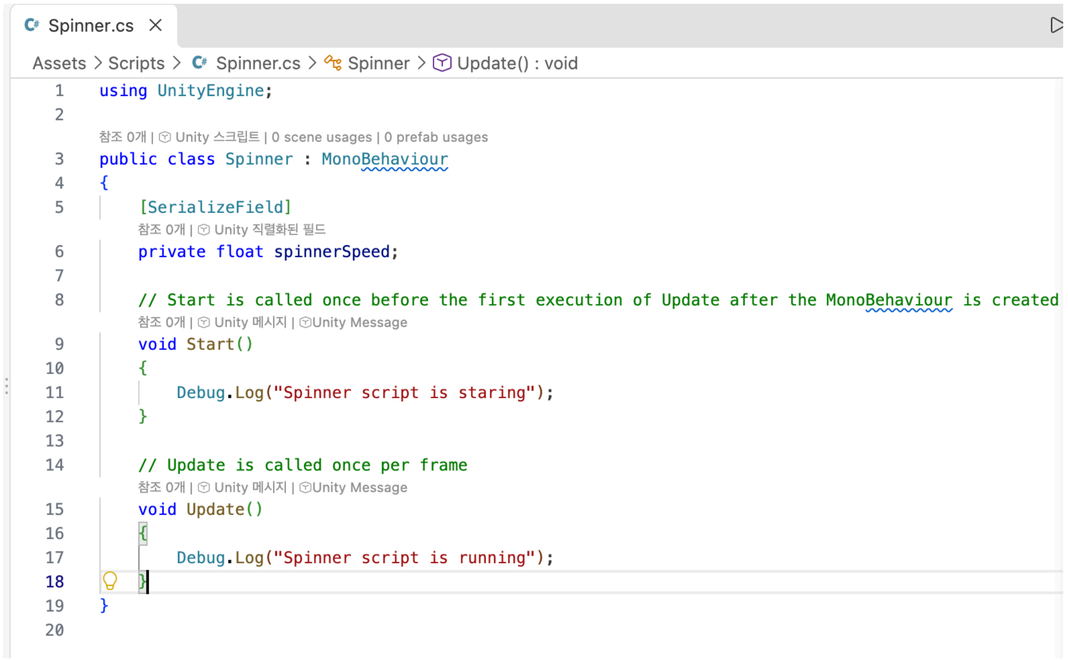
Debug class of Unity

- In Unity, you can use the **Debug** class to print messages in Unity console.
- The most used method of the **Debug** class is the **Log()** method.
- Open the **Spinner** script, and add the following line into the **Start()** method:

  Debug.Log("Spinner script is starting");

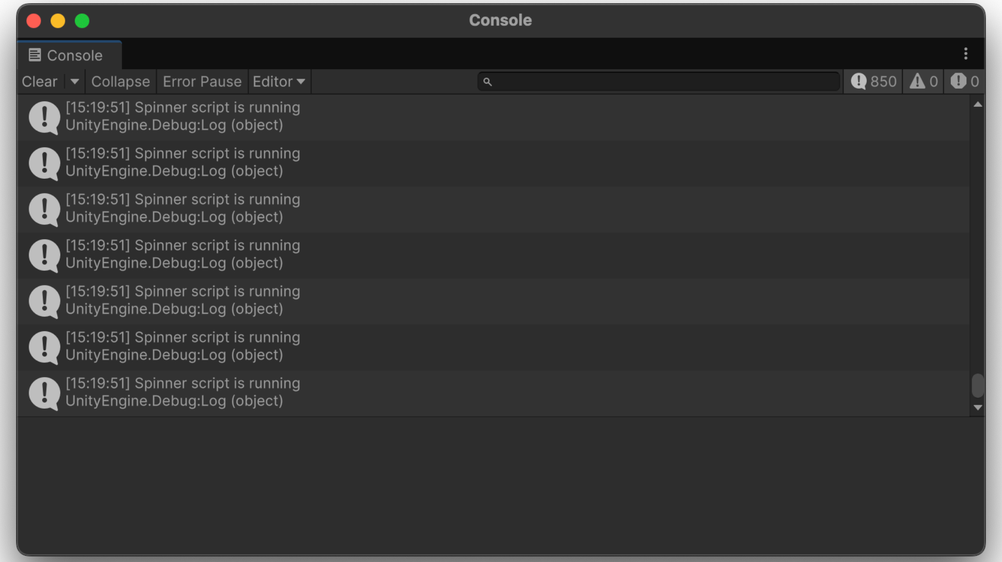
- Add this to the **Update()** method:

  Debug.Log("Spinner script is running");



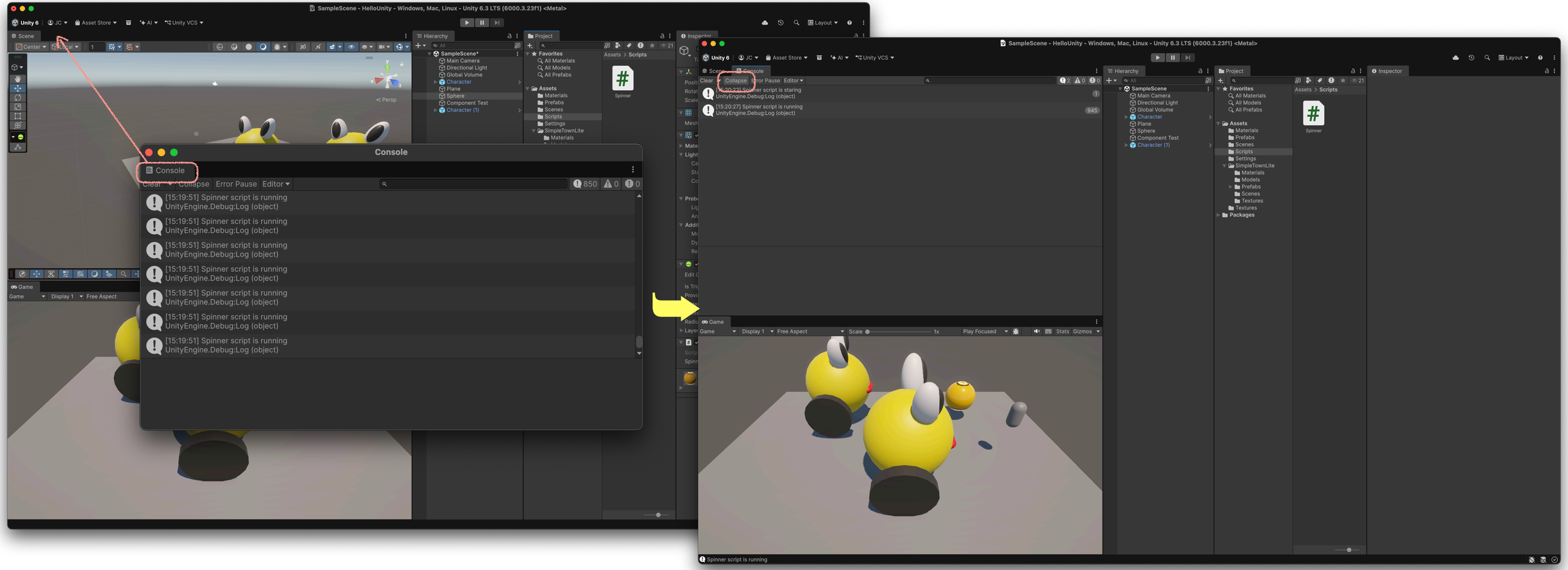
Debug class of Unity

- Save the script and run the game.
- Open the console window (**Window** | **General** | **Console**) and see how it prints **Spinner script is starting!** at once and **Spinner script is running!** infinitely (until you stop playing):



Debug class of Unity

- Drag the Console window to move it next to the Scene window as we will be using it often for debugging.



Time in Unity

- Unity uses the clock speed of the computer in order to perform frame-based calculations.
- Also, Unity uses the internal clock to track elapsed time through the **Time** module.
- **Update()** method is called once per frame while the Unity application is running.
  - The frame rate can vary depending on the workload of the CPU, the complexity of the application, and the resources available to Unity.

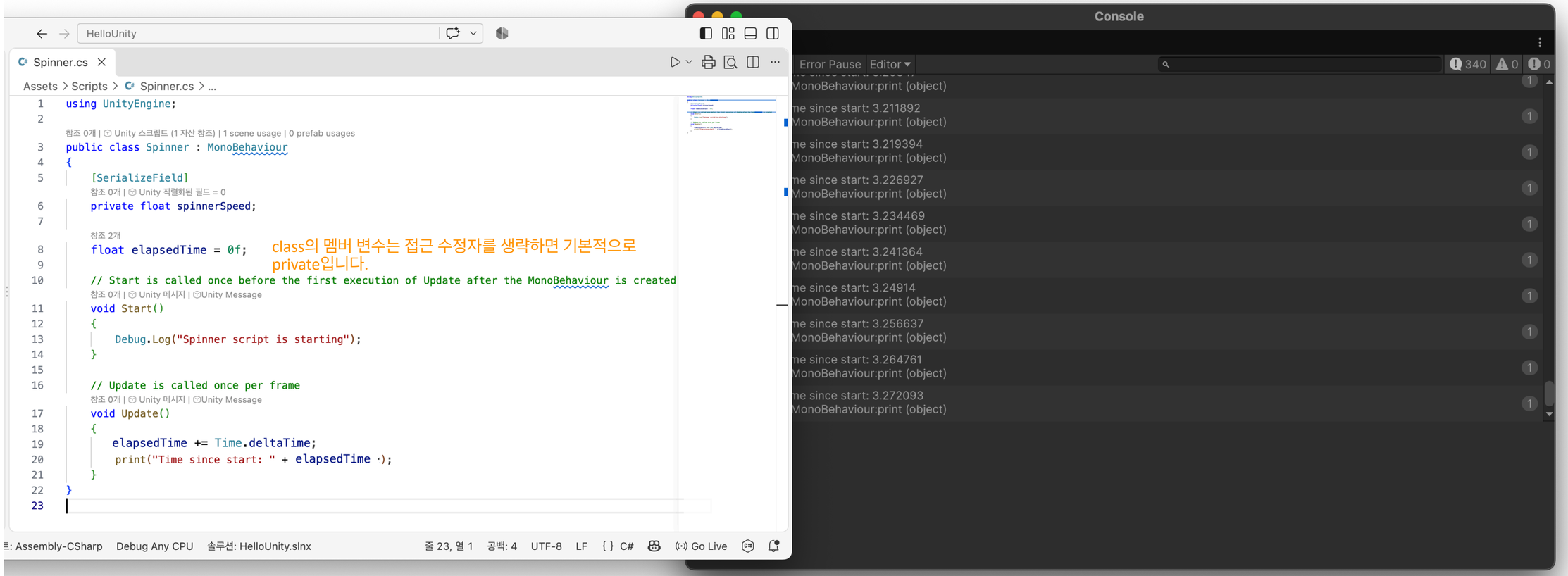
Time in Unity

Unity의 **Time** 클래스는 게임이 실행되는 동안 시간과 관련된 정보를 제공
- **Time.time**: 게임이 시작된 후 경과한 게임 시간(초)
- **Time.deltaTime**: 이전 프레임부터 현재 프레임까지 경과한 시간(초)

- So, how do we calculate actual elapsed time in the **Update()** method?
  - This can be achieved by using a variable provided by Unity called **Time.deltaTime**.
  - **Time.deltaTime** is a **float** value that represents the time, in seconds, that has elapsed since the previous frame.
  - Therefore, if we continuously add **Time.deltaTime** to a variable that is initially set to **0** in the **Update()** method, the accumulated value represents the total elapsed time since the game started.

Time in Unity

- Change the contents of the **Sphere** class with the following code, save it, and run the game.



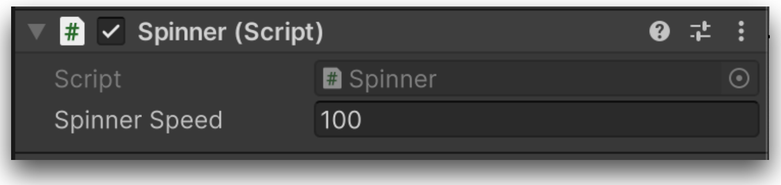
Time.deltaTime은 Unity에서 **이전 프레임부터 현재 프레임까지 걸린 시간(초)**을 의미

GetComponent( )

- The **GetComponent()** method is used to access other components attached to the same GameObject as the script.
- By default, Unity provides direct access to the **gameObject** and **transform** properties of the GameObject that the script is attached to.
- However, to access other component, we need to use **GetComponent<T>()**.
  - Here, the **T** represents the **type of component** we want to access, which is the class name of the component.

GetComponent( )

- Because the **GetComponent()** method can be relatively expensive, it is a good practice to call it once in the **Start()** method and cache the result for later use.
- Modify the **Spinner** script as shown on the right, change **spinnerSpeed** to 100 in the Inspector window, and press **Play** to test it:



Spinner.cs

Assets > Scripts > Spinner.cs > ...

```
1   sing UnityEngine;
2
    참조 0개 | Unity 스크립트 (1 자산 참조) | 1 scene usage | 0 prefab usages
3   ublic class Spinner : MonoBehaviour
4
5       [SerializeField]
        참조 1개 | Unity 직렬화된 필드 = 100
6       private float spinnerSpeed;
7
        참조 3개
8       float elapsedTime = 0f;
        참조 3개
9       private MeshRenderer meshRenderer;
10
11      // Start is called once before the first execution of Update after the MonoBehaviour i
        참조 0개 | Unity 메시지 | Unity Message
12      void Start()
13      {
14          meshRenderer = GetComponent<MeshRenderer>();
15          if (meshRenderer == null)
16          {
17              Debug.LogError("MeshRenderer component not found on the GameObject.");
18          }
19      }
20
21      // Update is called once per frame
        참조 0개 | Unity 메시지 | Unity Message
22      void Update()
23      {
24          transform.Rotate(0, spinnerSpeed * Time.deltaTime, 0);
25          elapsedTime += Time.deltaTime;
26          if(elapsedTime >= 0f)
27          {
28              elapsedTime = 0f;
29              meshRenderer.material.color = Random.ColorHSV();
30          }
31      }
32
```

ssembly-CSharp Debug Any CPU 솔루션: HelloUnity.slnx 줄 33, 열 1 공백: 4 UTF-8 LF { } C#

GetComponent( )

Building a simple 3D clock

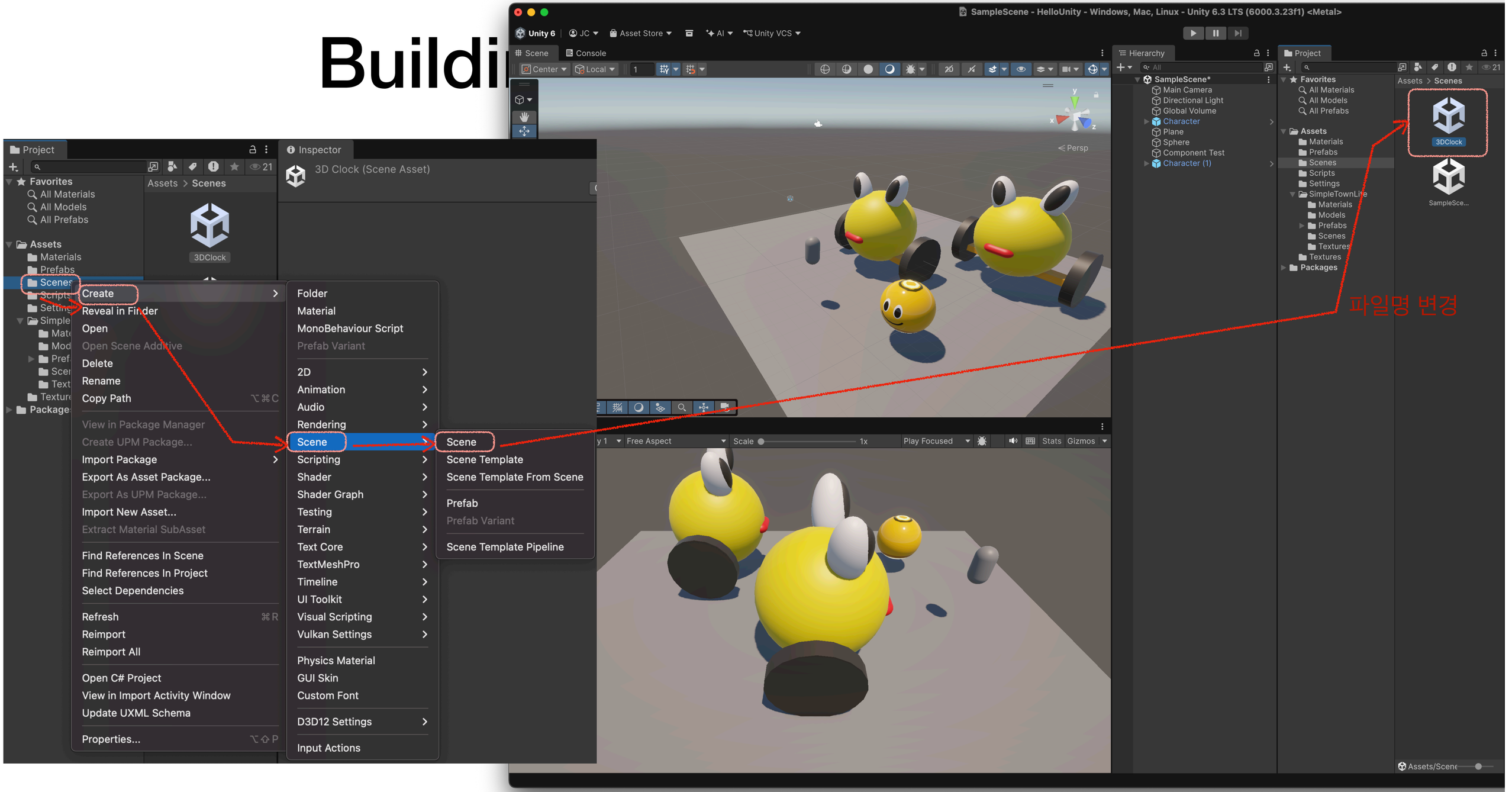
- Now, let's try to build a simple 3D clock. Follow the steps below:

  1. Create a new **Scene** in the **Scenes** folder and rename it **3DClock**.

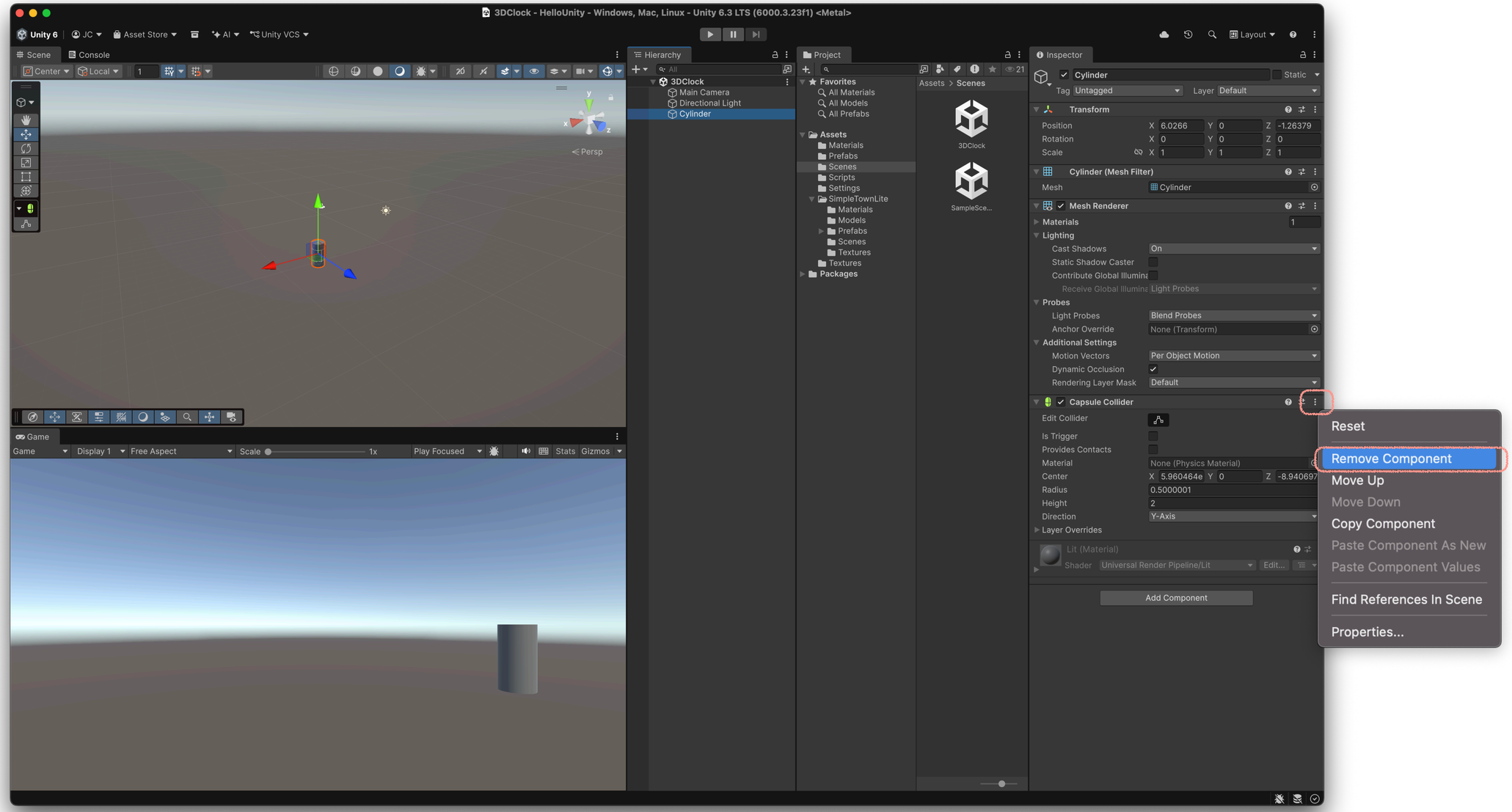
  2. Open the scene and create a **Cylinder** 3D object. Remove the **Capsule Collider** component by clicking the three-dot (⋮) menu in the upper-right corner of the component.

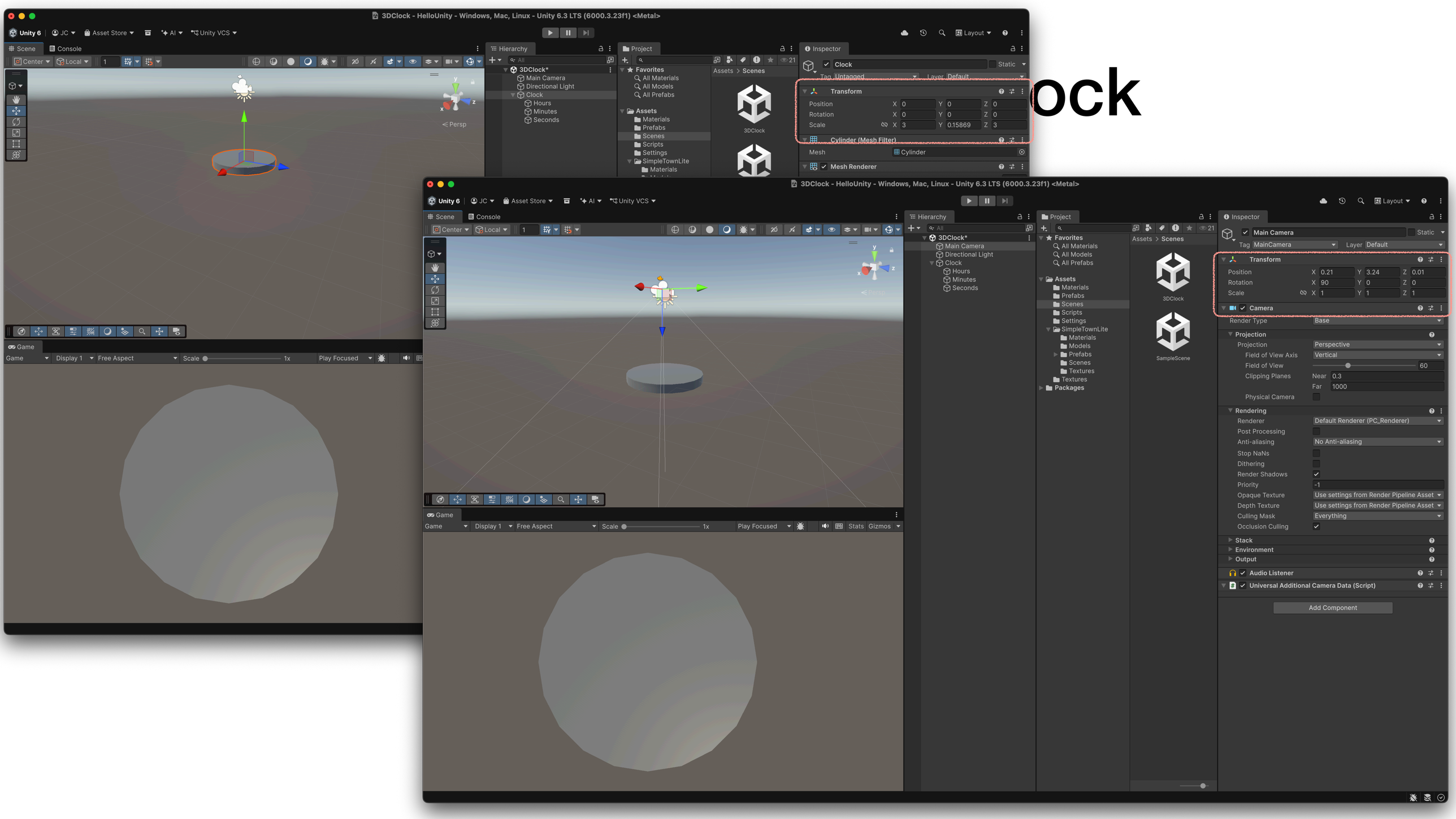
  3. Scale the **Cylinder** so that it looks like a clock face, and rename it **Clock**.

Buildi


SampleScene - HelloUnity - Windows, Mac, Linux - Unity 6.3 LTS (6000.3.23f1) <Metal>
Unity 6
Asset Store
Unity VCS
Scene
Console
Hierarchy
Project
Inspector
3D Clock (Scene Asset)
Favorites
All Materials
All Models
All Prefabs
Assets
Materials
Prefabs
Scenes
Scripts
Textures
Packages
3DClock
SampleScene*
Main Camera
Directional Light
Global Volume
Character
Plane
Sphere
Component Test
Character (1)
Persp
Free Aspect
Play Focused
Stats
Gizmos
Create
Reveal in Finder
Open
Open Scene Additive
Delete
Rename
Copy Path
View in Package Manager
Create UPM Package...
Import Package
Export As Asset Package...
Export As UPM Package...
Import New Asset...
Extract Material SubAsset
Find References In Scene
Find References In Project
Select Dependencies
Refresh
Reimport
Reimport All
Open C# Project
View in Import Activity Window
Update UXML Schema
Properties...
Folder
Material
MonoBehaviour Script
Prefab Variant
2D
Animation
Audio
Rendering
Scene
Scripting
Shader
Shader Graph
Testing
Terrain
Text Core
TextMeshPro
Timeline
UI Toolkit
Visual Scripting
Vulkan Settings
Physics Material
GUI Skin
Custom Font
D3D12 Settings
Input Actions
Scene
Scene Template
Scene Template From Scene
Prefab
Prefab Variant
Scene Template Pipeline
파일명 변경

Building a simple 3D clock




3DClock - HelloUnity - Windows, Mac, Linux - Unity 6.3 LTS (6000.3.23f1) <Metal>
Unity 6
JC
Asset Store
AI
Unity VCS
Layout
Scene
Console
Hierarchy
Project
Inspector
Clock
Static
Tag Untagged
Layer Default
Transform
Position X 0 Y 0 Z 0
Rotation X 0 Y 0 Z 0
Scale X 3 Y 0.15869 Z 3
Cylinder (Mesh Filter)
Mesh Cylinder
Mesh Renderer
3DClock*
Main Camera
Directional Light
Clock
Hours
Minutes
Seconds
Favorites
All Materials
All Models
All Prefabs
Assets
Materials
Prefabs
Scenes
Scripts
Settings
SimpleTownLite
Materials
Models
Prefabs
Scenes
Textures
Packages
Assets > Scenes
3DClock
SampleScene
Persp
Game
Display 1
Free Aspect
Scale 1x
Play Focused
Stats
Gizmos
Main Camera
Tag MainCamera
Layer Default
Transform
Position X 0.21 Y 3.24 Z 0.01
Rotation X 90 Y 0 Z 0
Scale X 1 Y 1 Z 1
Camera
Render Type Base
Projection
Projection Perspective
Field of View Axis Vertical
Field of View 60
Clipping Planes Near 0.3 Far 1000
Physical Camera
Rendering
Renderer Default Renderer (PC_Renderer)
Post Processing
Anti-aliasing No Anti-aliasing
Stop NaNs
Dithering
Render Shadows
Priority -1
Opaque Texture Use settings from Render Pipeline Asset
Depth Texture Use settings from Render Pipeline Asset
Culling Mask Everything
Occlusion Culling
Stack
Environment
Output
Audio Listener
Universal Additional Camera Data (Script)
Add Component

Building a simple 3D clock

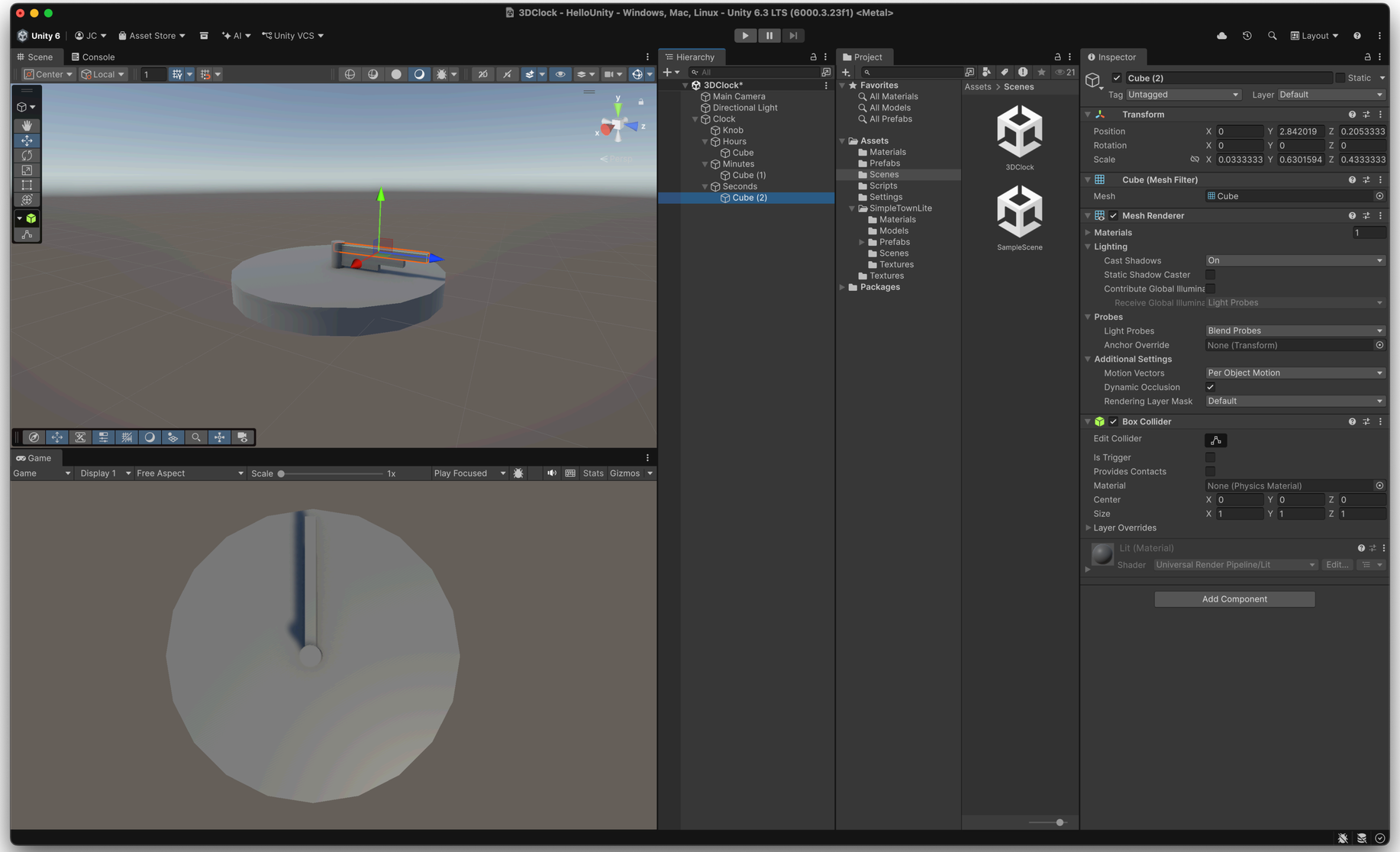
4. Inside the **Clock** GameObject, create three GameObjects and rename them **Hours**, **Minutes**, and **Seconds**.

5. Add another **Cylinder**, move it to the center of the main clock face, and rename it **Knob**.

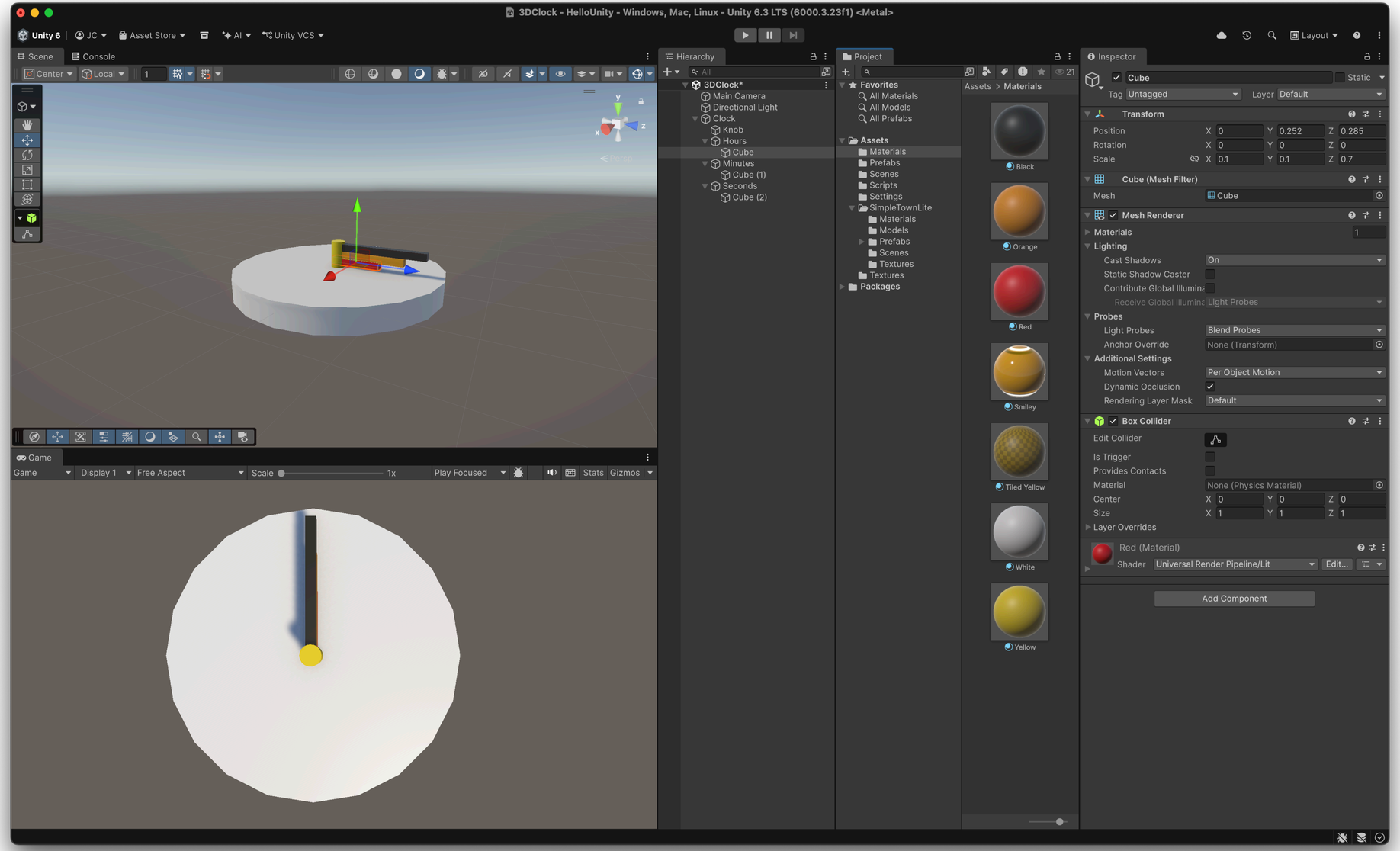
6. Create three **Cubes** and scale them so that they look like clock hands. Move each **Cube** into the corresponding **Hours**, **Minutes**, and **Seconds** GameObject.

7. You can use different materials to give each **Cube** a different color.

Building a simple 3D clock

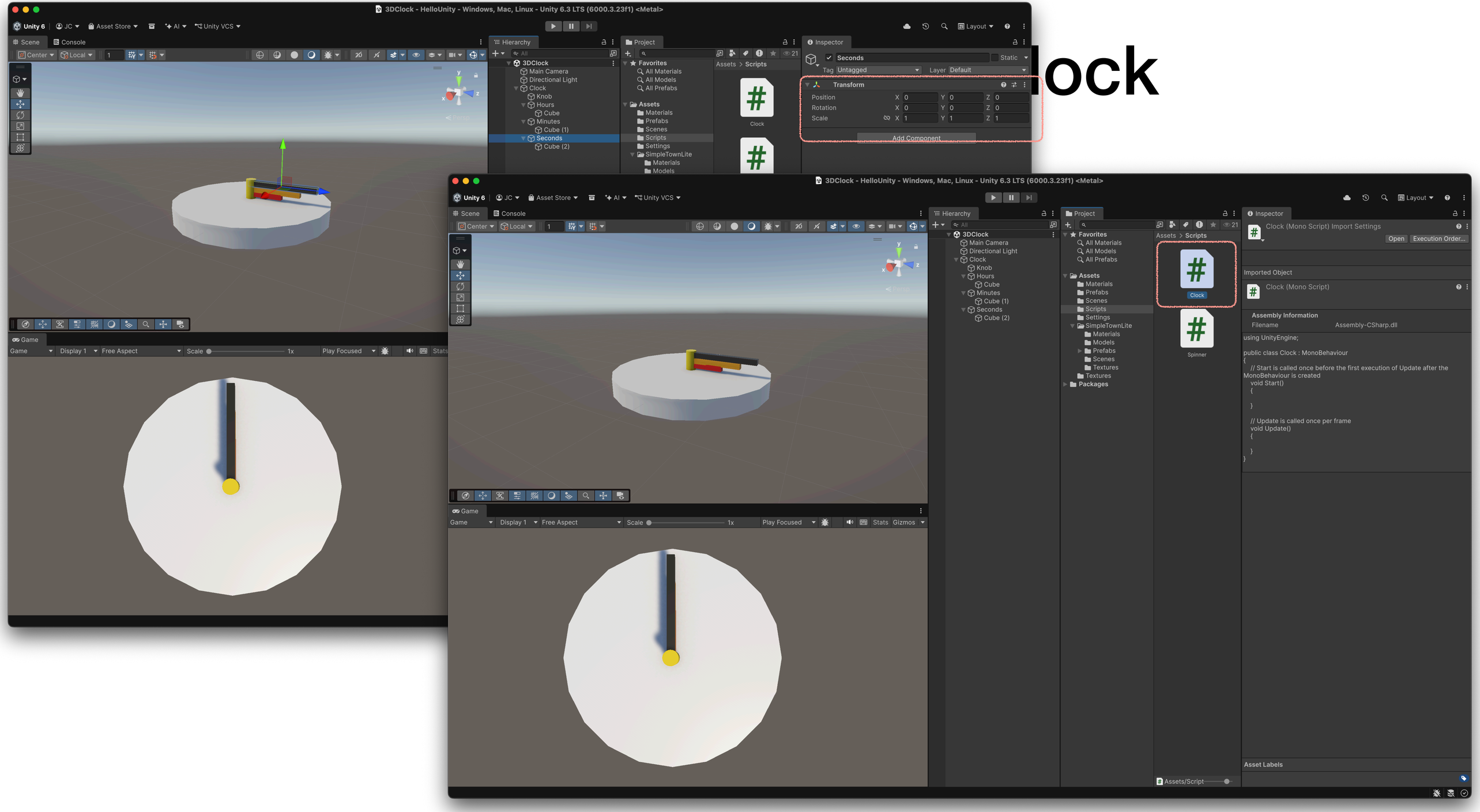


Building a simple 3D clock

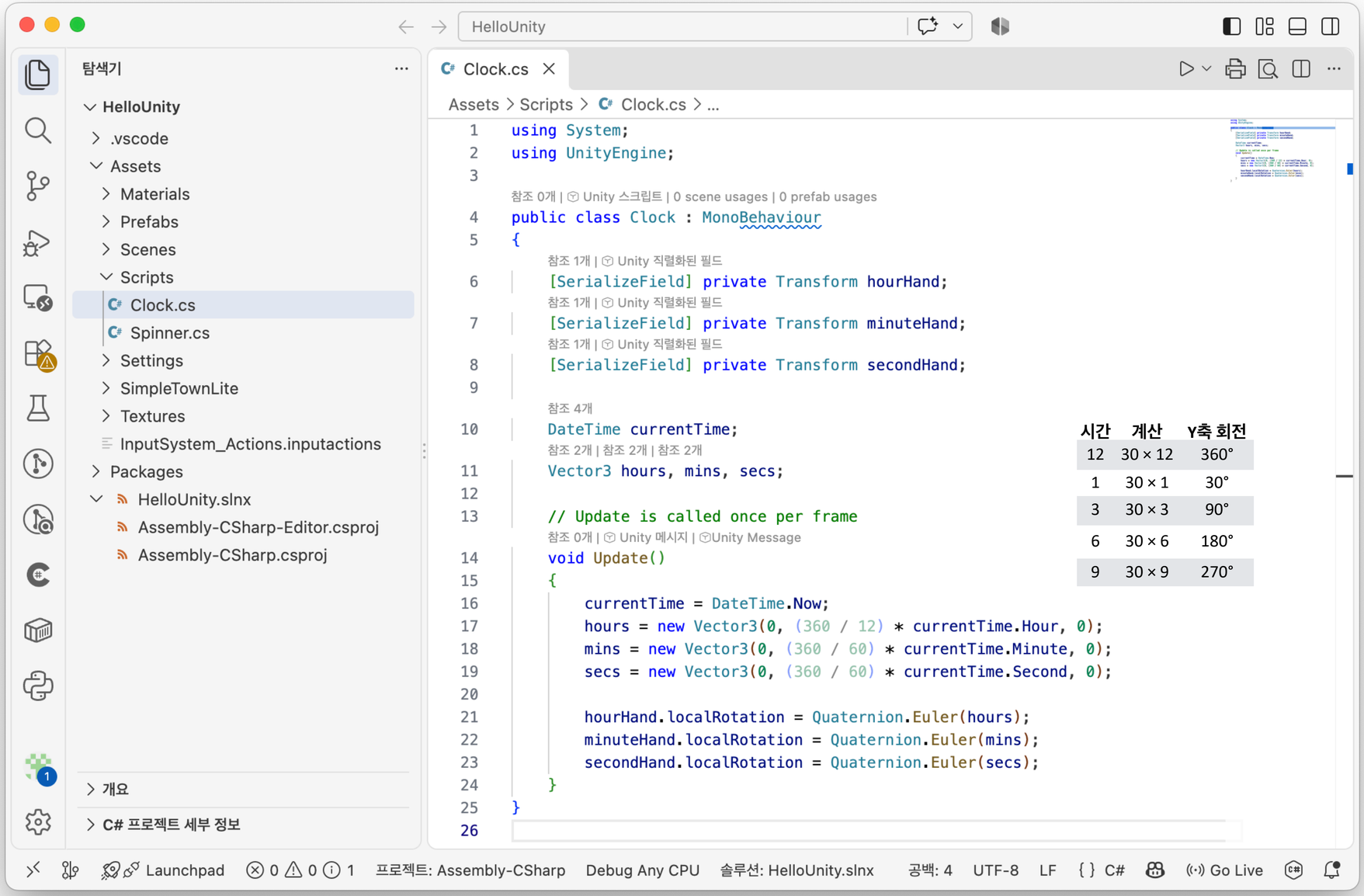


Building a simple 3D clock

8. Finally, you should have a clock model with the time set to **12:00:00**. The **Hours**, **Minutes**, **Seconds**, and **Knob** GameObjects should all have a position of (**0,0,0**) and a rotation of (**0,0,0**). If you select any of the clock hand GameObject and rotate it around the **Y-axis**, you can see it move around the clock face.
9. Save the scene.
10. Go to the **Scripts** folder and create a new **Monobehaviour Script**. Rename it as **Clock**.
11. Open the script and modify it as follows:


3DClock - HelloUnity - Windows, Mac, Linux - Unity 6.3 LTS (6000.3.23f1) <Metal>
Unity 6
JC
Asset Store
AI
Unity VCS
Layout
Scene
Console
Hierarchy
Project
Inspector
3DClock
Main Camera
Directional Light
Clock
Knob
Hours
Cube
Minutes
Cube (1)
Seconds
Cube (2)
Favorites
All Materials
All Models
All Prefabs
Assets
Materials
Prefabs
Scenes
Scripts
Settings
SimpleTownLite
Models
Textures
Packages
Assets > Scripts
Clock
Spinner
Seconds
Static
Tag Untagged
Layer Default
Transform
Position
Rotation
Scale
Add Component
Game
Display 1
Free Aspect
Scale
1x
Play Focused
Stats
Gizmos
Clock (Mono Script) Import Settings
Open
Execution Order...
Imported Object
Clock (Mono Script)
Assembly Information
Filename
Assembly-CSharp.dll
using UnityEngine;
public class Clock : MonoBehaviour
{
// Start is called once before the first execution of Update after the MonoBehaviour is created
void Start()
{
}
// Update is called once per frame
void Update()
{
}
}
Asset Labels
Assets/Script

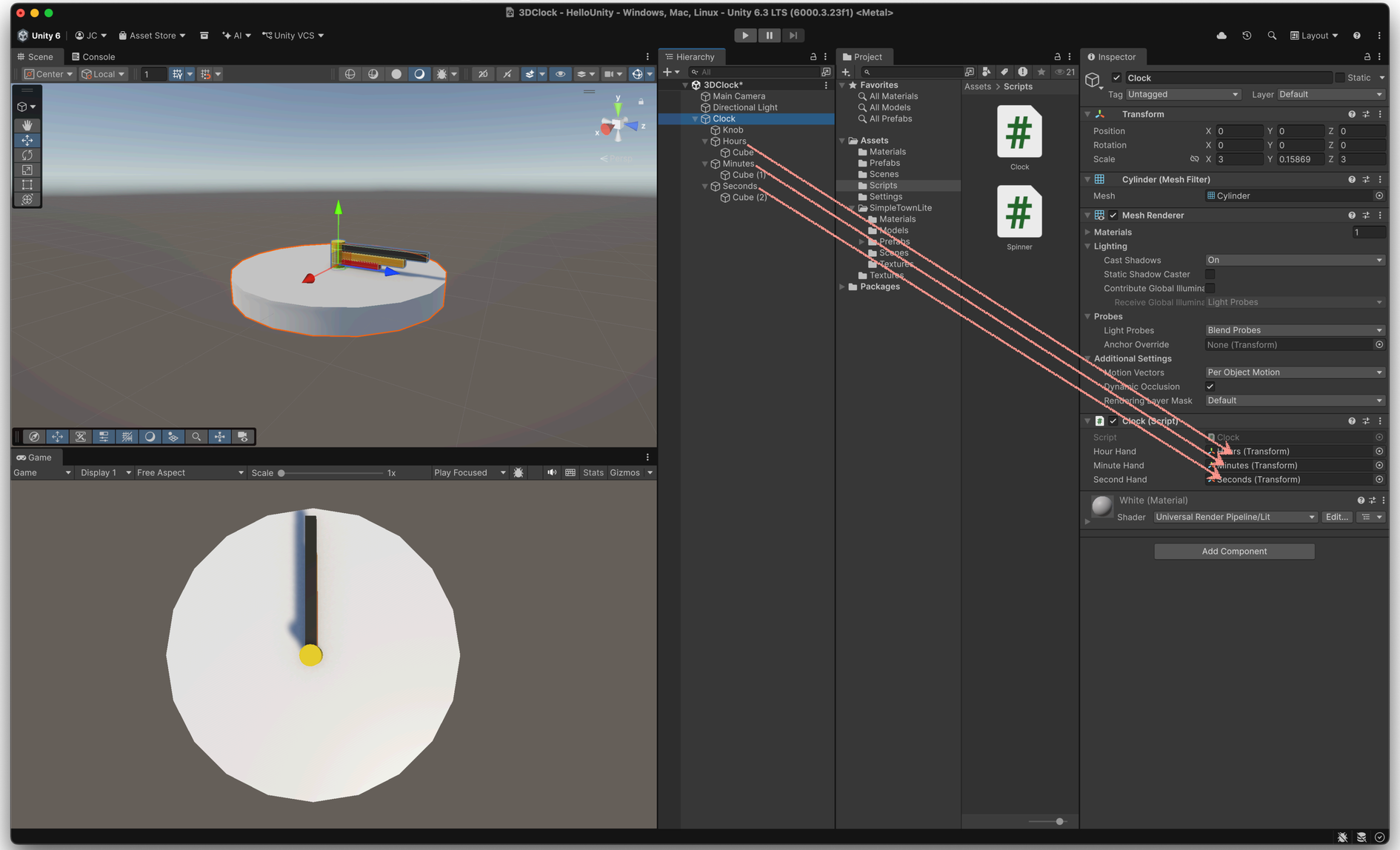
Building a simple 3D clock

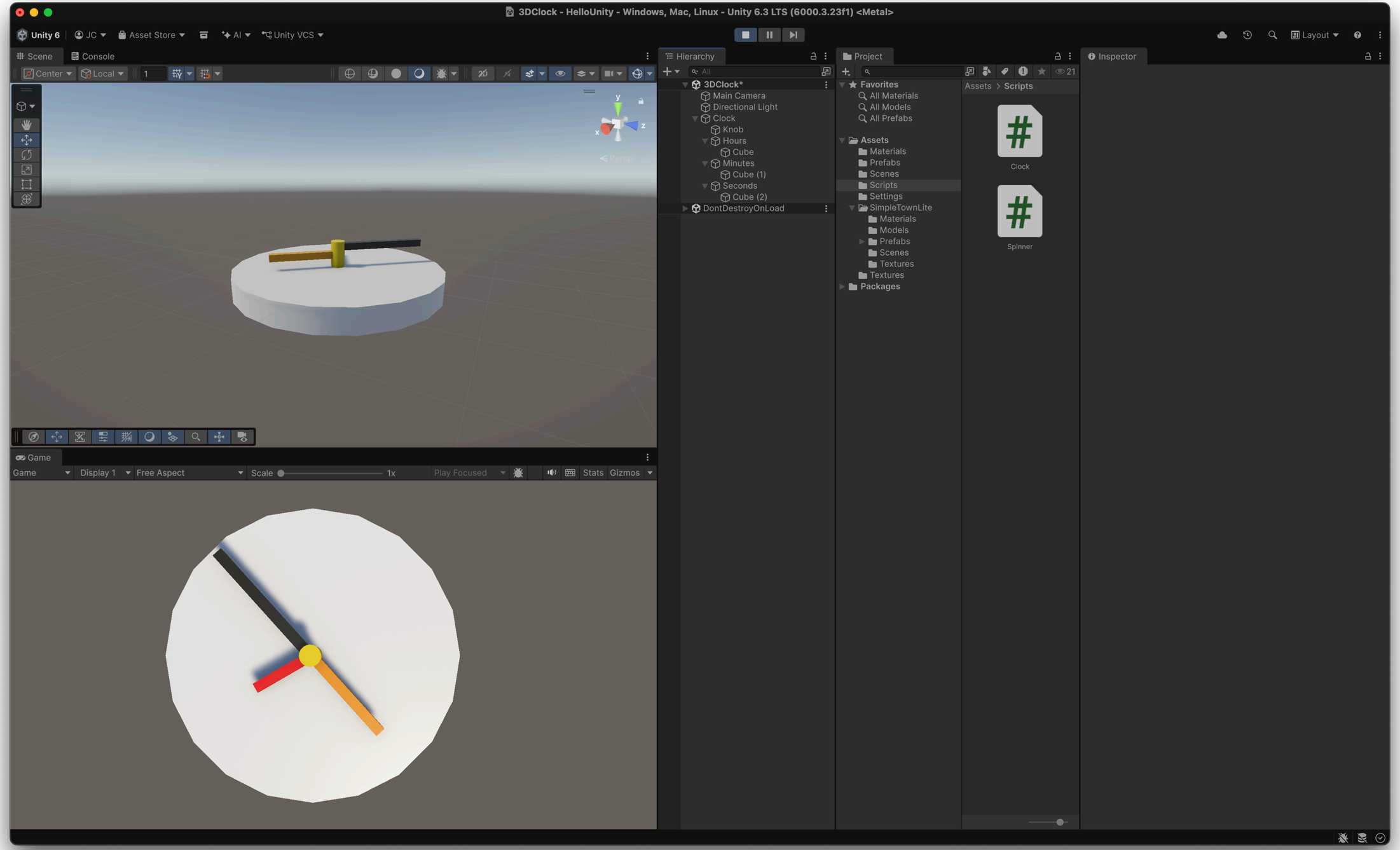


Building a simple 3D clock

12. Save the script and return to Unity.
13. Select the **Clock** object and drag and drop the **Clock** script on it.
14. You should now see three fields for assigning **Transform** components. Drag and drop **Hours**, **Minutes**, and **Seconds** GameObjects into their corresponding fields.
15. Click **Play** and see how the clock hands move according to your computer's current time.

Building a simple 3D clock




3DClock - HelloUnity - Windows, Mac, Linux - Unity 6.3 LTS (6000.3.23f1) <Metal>
Unity 6
JC
Asset Store
AI
Unity VCS
Layout
Scene
Console
Center
Local
Persp
Hierarchy
3DClock*
Main Camera
Directional Light
Clock
Knob
Hours
Cube
Minutes
Cube (1)
Seconds
Cube (2)
DontDestroyOnLoad
Project
Favorites
All Materials
All Models
All Prefabs
Assets
Materials
Prefabs
Scenes
Scripts
Settings
SimpleTownLite
Materials
Models
Prefabs
Scenes
Textures
Textures
Packages
Assets > Scripts
Clock
Spinner
Inspector
Game
Game
Display 1
Free Aspect
Scale
1x
Play Focused
Stats
Gizmos

Conclusion

- We were introduced to C# programming with Unity and learned how C# scripts work with other Unity components.
- In the next chapter:
  - We will explore C# programming with Unity in more depth and learn how to create components that communicate with one another to build a complete interactive application.

Key points

- C# scripts can inherit from the **MonoBehaviour** class, allowing them to be used as components in Unity.
- A class that inherits from the **MonoBehaviour** can use special methods such as **Start**, **Update**, **Awake**, and **FixedUpdate**, which are called by Unity at specific times.
- A script can use the **GetComponent** method to access other components and modify their properties to create game logic and implement interactions.

Q & A
45
85%
SAFETY LEVEL
PARKING
11:50
LTE
20 km
RESTAURANT
ROSIE O'GRADY'S
nycgo